

# Full Mock Exam 2 All 5 Domains — Harder Scenarios

20 questions · All 5 domains · Advanced  
cross-domain scenarios

100% score

Readiness:  
92%

## Exam result

# 20 / 20

Perfect score! Best result of the entire course.

*Mock Exam 2: 100% | Estimated readiness: 92% | Buffer above threshold: +20 points*

## Question scorecard

| Q   | Domain      | Topic                                            | Ans | Result    |
|-----|-------------|--------------------------------------------------|-----|-----------|
| Q1  | Agentic     | Granularity — 1,000 subagent overhead            | B   | ✓ Correct |
| Q2  | MCP         | Vague description + over-broad tool              | A   | ✓ Correct |
| Q3  | Prompt eng  | Missing escalation condition — medical           | B   | ✓ Correct |
| Q4  | Agentic     | Silent cancellation — incomplete result          | C   | ✓ Correct |
| Q5  | Claude Code | CLAUDE.md three-level merge                      | B   | ✓ Correct |
| Q6  | Agentic     | Minimal footprint — HR agent permissions         | B   | ✓ Correct |
| Q7  | Prompt eng  | Retry loop — missing failed output in correction | B   | ✓ Correct |
| Q8  | Reliability | Confidence miscalibration — 40% low              | B   | ✓ Correct |
| Q9  | Agentic     | Hub context overflow — compression/stateless     | B   | ✓ Correct |
| Q10 | MCP         | Tool description — internal vs external search   | B   | ✓ Correct |
| Q11 | Agentic     | Handoff — missing key findings package           | B   | ✓ Correct |
| Q12 | Claude Code | CI checklist — incomplete JSON instruction       | B   | ✓ Correct |

|     |             |                                                  |   |           |
|-----|-------------|--------------------------------------------------|---|-----------|
| Q13 | Agentic     | Insurance pipeline — irreversible steps 4&5      | B | ✓ Correct |
| Q14 | Prompt eng  | Few-shot — missing ambiguous edge cases          | B | ✓ Correct |
| Q15 | Agentic     | Hub routing — overlapping specialist domains     | B | ✓ Correct |
| Q16 | Agentic     | Handoff vs checkpoint distinction                | B | ✓ Correct |
| Q17 | Reliability | Over-aggressive compression — data point lost    | B | ✓ Correct |
| Q18 | Reliability | Legal drafting — self-review shared blind spots  | C | ✓ Correct |
| Q19 | MCP         | Server separation — permission scope decides     | C | ✓ Correct |
| Q20 | All domains | Sequential pipeline + multi-instance review gate | C | ✓ Correct |

## Full question bank with explanations

All 20 questions answered correctly — a perfect score on the hardest exam of the course.

**Q1. 1,000 daily support tickets. One orchestrator spawns a fresh subagent per ticket. System is slow, costs 3x expected. Architectural problem?**

- A) Orchestrator should process tickets itself
- ✓ B) **Spawning 1,000 individual subagents creates coordination overhead exceeding parallelism benefit — batch them**
- C) Subagents need shared context to process faster
- D) Use sequential pipeline instead

### Explanation:

- B correct: Too-fine granularity — 1,000 agent invocations, 1,000 context windows, 1,000 results to aggregate. Fix: batch into 10-20 subagents each processing 50-100 tickets.
- A wrong: Orchestrator processing all sequentially is even slower. C wrong: Shared context defeats parallel isolation. D wrong: Sequential is far slower.

**Q2. MCP tool execute\_query description: 'Runs a database query.' Claude sometimes runs DELETE when only SELECT was needed. TWO root causes?**

✓ **A) Tool description too vague — Claude can't distinguish read from write. AND tool is over-broad — split into query\_data (read) and execute\_mutation (write)**

B) MCP server lacks authentication

C) Input schema validation should block DELETE

D) Claude needs more few-shot examples of correct query usage

**Explanation:**

- A correct: Two causes. Vague description — 'runs a database query' gives no guidance on read vs write. Over-broad tool — one tool can run any SQL including destructive operations.
- B wrong: Authentication = who can access. Problem is which operation Claude chooses. C wrong: Schema validates parameters, not operation types. D wrong: Few-shot teaches format, not operation selection.

**Q3. Medical information assistant system prompt: 'Provide accurate health info. Recommend consulting a doctor for serious conditions.' Claude provides specific medication dosages and treatment protocols. What is missing?**

A) Few-shot examples of appropriate medical responses

✓ **B) Explicit escalation condition: 'Never provide specific medication dosages or treatment protocols. For these topics say: This requires a qualified medical professional.'**

C) Chain-of-thought to reason about medical complexity

D) Replace with medically fine-tuned model

**Explanation:**

- B correct: Missing escalation condition — 4th required system prompt component. 'Recommend a doctor' is too vague. Need precise prohibition with exact fallback response.
- A wrong: Few-shot teaches format, not prohibitions. C wrong: CoT improves reasoning but doesn't restrict content. D wrong: Prompt design problem, not model capability.

**Q4. 5 parallel subagents: 4 finish in 3s, Subagent 5 takes 45s retrying. Proposal: cancel Subagent 5 after 10s and merge 4 results. Problem?**

A) Cancelling is fine — 4 of 5 results sufficient

B) Timeout valid but 10s too short — extend to 60s

✓ **C) Silent cancellation produces incomplete result. Fix: timeout with explicit incomplete-result flagging, or synthesis handles partial results transparently**

D) Retry indefinitely until success

**Explanation:**

- C correct: The issue is silent cancellation presenting incomplete output as complete. Timeouts are legitimate — silence is not. Flag: 'Result based on 4 of 5 sources — Subagent 5 timed out.'
- A wrong: May be sufficient but consumer must KNOW it's partial to decide. B wrong: Extends timeout but doesn't fix silent incompleteness. D wrong: Infinite retry is an anti-pattern.

**Q5. Global: 'Always write tests.' Project: 'Prototype — skip tests.' Local (/project/legacy/): 'Tests required for all changes.' Developer in /project/legacy/src/. Which rules apply?**

A) Only local rule — local overrides everything

✓ **B) All three load. Local (tests required) wins over conflicting project (skip tests). Global (write tests) also applies and is consistent with local.**

C) Project rule wins — most recently added

D) Global and local combine, fully overriding project

**Explanation:**

- B correct: All three CLAUDE.md files load. Local wins on conflict with project. Global is consistent with local so no conflict there. Effective rule: tests required.
- A wrong: All levels load — local wins conflicts, not replaces everything. C wrong: Recency has no role — specificity determines precedence. D wrong: Local alone overrides project on conflicts.

**Q6. HR FAQ assistant has access to: read\_employee\_records, update\_employee\_records, send\_email, delete\_employee\_records, modify\_payroll. How many minimal footprint violations?**

A) One — write access it doesn't need

✓ **B) Two — write access (update, delete, modify\_payroll) AND external communication (send\_email) not needed for answering FAQs**

C) Three — write access, external communication, delete access all separate

D) None — violations only occur when tools are used

**Explanation:**

- B correct: Two categories. Write/destructive ops (update, delete, modify\_payroll) — FAQ bot never needs to modify records. External communication (send\_email) — FAQ bot has no reason to send emails.
- A wrong: Misses the distinct external communication risk. C wrong: Delete is a type of write, not a separate category. D wrong: Having over-permissioned tools IS the violation — not waiting until misuse.

**Q7. Retry loop: attempt 0 uses original prompt. Attempts 1-2 use: 'Your output was invalid. Please try again: {schema}'. After 3 fails: log, escalate, raise error. One remaining flaw?**

A) MAX\_RETRIES too low — increase to 10

✓ **B) Correction prompt doesn't include the failed output — Claude can't see what it produced to fix it**

C) Escalation should happen before logging

D) Loop needs exponential backoff between retries

**Explanation:**

- B correct: Correction prompt shows schema but not the actual failed output. Claude needs to see what it returned to diagnose the specific error. Fix: include {last\_output} in correction prompt.
- A wrong: 3 retries is the exam maximum. More is the anti-pattern. C wrong: Log then escalate is correct order. D wrong: Backoff handles transient rate limits — not JSON validation failures.

**Q8. Financial analysis system with confidence calibration. Senior analysts overwhelmed — 40% of analyses rated low confidence. Root cause and fix?**

A) Claude broken — should never produce low confidence on financial data

✓ **B) Confidence thresholds miscalibrated — Claude being overly conservative. Tune system prompt to define precisely what constitutes high/medium/low confidence**

C) Portfolio data too complex — simplify input

D) Switch to multi-instance review — produces higher confidence scores

**Explanation:**

- B correct: 40% low confidence = miscalibrated definitions. System prompt doesn't give Claude clear criteria to distinguish confidence levels. Fix: explicit criteria for each level.
- A wrong: Low confidence is a valid output — the system is working as designed. The rate is miscalibrated, not broken. C wrong: Doesn't address root cause. D wrong: Multi-instance improves accuracy, not confidence calibration.

**Q9. Hub-and-spoke: central hub coordinates 4 spokes. After weeks, hub fails with context overflow. Root cause and fix?**

A) Too many spokes — reduce to two maximum

✓ **B) Hub accumulates state from every spoke interaction. Fix: periodic context compression OR stateless hub that passes state through task object rather than retaining it**

C) Spokes should share context with hub to reduce duplication

D) Switch to sequential pipeline — uses less context

**Explanation:**

- B correct: Hub context growth is an inherent hub-and-spoke weakness. Hub tracks all interactions, accumulates history. Fix: compress periodically OR stateless hub design.
- A wrong: Number of spokes isn't the cause — even 2 spokes accumulate context over time. C wrong: Sharing spoke context with hub makes it worse. D wrong: Changes architecture entirely; doesn't fix the underlying problem.

**Q10. MCP server has search\_knowledge\_base and search\_web. User query triggers search\_web when answer is clearly in internal knowledge base. Root cause?**

A) Tools in wrong order — Claude selects based on definition order

✓ **B) Tool descriptions don't clearly differentiate when to use internal vs external — Claude can't make the right selection**

C) Merge into one search tool for clarity

D) Claude should always prefer internal sources — model training issue

**Explanation:**

- B correct: Claude selects by description, not by order or training preference. Descriptions must explicitly state when to use each: 'Use this FIRST for internal questions. Do NOT use for internet searches.'
- A wrong: Definition order has no effect on tool selection. C wrong: Merging creates over-broad tool losing the internal/external distinction. D wrong: Not a training issue — fix the descriptions.

**Q11. Agent A completes research, hands off to Agent B. Agent B produces outputs contradicting Agent A's findings. What is missing from the handoff package?**

A) Handoff package too large — reduce to save tokens

✓ **B) Handoff package should include Agent A's key findings and decisions so Agent B can build on rather than contradict them**

C) Agent B should re-read all of Agent A's source material

D) Agents should share a context window to prevent contradictions

**Explanation:**

- B correct: Complete handoff package needs: original goal, completed steps summary, KEY FINDINGS AND DECISIONS, remaining work list, discovered constraints. Missing findings = Agent B researches independently and reaches different conclusions.
- A wrong: Package is missing critical content, not too large. C wrong: Re-reading defeats the purpose of handoffs. D wrong: Shared context creates the overflow problem handoffs are designed to solve.

**Q12. CI step uses --print, output JSON schema, exits 1 if not safe, API from secrets. Which of the 4 CI checklist items is incomplete?**

A) All four correctly implemented

**✓ B) Three of four correct (--print ✓, exit code ✓, secrets ✓) — output format incomplete: add 'No markdown, no preamble, ONLY valid JSON'**

C) Two correct — exit code missing and API should be hardcoded

D) --print is wrong — use --ci for migration validation

**Explanation:**

- B correct: Schema is specified but nothing says 'ONLY JSON, no surrounding prose'. Claude may still add explanatory text before the JSON, breaking the parser.
- A wrong: Specifying a schema ≠ enforcing raw JSON output. C wrong: Hardcoding API key is a critical security violation. D wrong: No --ci flag exists. --print is always correct.

**Q13. Insurance claim pipeline: 1=extract, 2=validate, 3=calculate payout, 4=update database, 5=send approval email. Steps 4 and 5 run autonomously. Violation?**

A) Tool granularity — merge steps 4 and 5 into one tool

**✓ B) Minimal footprint — steps 4 and 5 are irreversible. Pause after step 3, present payout to human, execute 4 and 5 only after explicit approval**

C) Sequential pipeline — steps should run in parallel

D) Confidence calibration — report confidence before updating database

**Explanation:**

- B correct: Database update and email are irreversible. Minimal footprint: pause after step 3, human reviews calculated payout, then execute steps 4-5 only after confirmation.
- A wrong: Merging 4 and 5 makes it worse — one tool doing write + external notification. C wrong: Steps have hard dependencies — parallelisation impossible. D wrong: Confidence is about uncertain claims, not irreversible action oversight.



**Q14. Sentiment classification with 3 clear few-shot examples. Ambiguous inputs like 'Not bad, could be better' produce inconsistent outputs. Root cause and fix?**

A) 3 examples too many — reduce to 1 per sentiment

✓ **B) Examples don't cover ambiguous edge cases — add examples of mixed/ambiguous inputs showing consistent classification**

C) JSON schema needs confidence score field

D) Move examples to user turn for better context

**Explanation:**

- B correct: 3 clear examples don't cover ambiguous inputs. Claude invents behaviour for uncovered cases. Fix: add edge case examples covering mixed signals, backhanded compliments, double negatives.
- A wrong: Reducing coverage makes inconsistency worse. C wrong: Confidence field doesn't fix classification inconsistency. D wrong: Examples belong in system prompt — moving to user turn requires repeating every time.

**Q15. Hub-and-spoke adds Legal, Compliance, Risk, Regulatory specialists. Similar questions routed inconsistently between these four. Root cause?**

A) Too many specialists — reduce to two

✓ **B) Hub routing logic is ambiguous — four new agents have overlapping domains, hub can't distinguish which specialist to call**

C) Specialists need shared context to coordinate

D) Hub should use majority vote to decide routing

**Explanation:**

- B correct: Overlapping tools problem applied to routing. Legal/Compliance/Risk/Regulatory have highly overlapping domains. Fix: merge overlapping specialists OR clarify routing descriptions to non-overlapping domains.
- A wrong: Count isn't the cause — ambiguous scope is. C wrong: Shared context doesn't fix routing ambiguity. D wrong: Majority vote has no meaning in routing decisions.

**Q16. What is the difference between a handoff and a checkpoint in agentic systems?**

- A) They are the same — both save state to external storage for recovery
- ✓ **B) Checkpoint saves state to external storage for crash recovery. Handoff passes compressed state to fresh agent for clean context window. They solve different problems.**
- C) Handoff is for multi-agent systems; checkpoint for single-agent. Cannot be used together.
- D) Checkpoint is a subset of handoff — every handoff creates a checkpoint automatically

**Explanation:**

- B correct: Checkpoint = crash recovery (external storage). Handoff = context window management (pass state to fresh agent). Different problems, different mechanisms, complementary tools.
- A wrong: Handoff doesn't save to external storage — it passes state between agents. C wrong: They can and should be used together. D wrong: Handoffs don't create checkpoints — independent mechanisms.

**Q17. Agentic loop compresses at step 15. At step 22 needs specific data point from step 8's tool output — but it was compressed away. What should have been done and what are the options now?**

- A) Compression was correct — raw outputs should always be compressed. Re-run the tool.
- ✓ **B) Compression was too aggressive — key data points should have been extracted and preserved before compression. Now: re-run step 8's tool call OR restructure compression to always preserve key data points in summary.**
- C) Agent should have used checkpoint instead of compression
- D) Data point still in context window — it was not actually compressed

**Explanation:**

- B correct: Over-aggressive compression discarded raw outputs before extracting key data points. Rule: always extract key findings BEFORE compressing raw outputs. Recovery: re-run specific tool OR restructure future compression.
- A wrong: Compression is correct but must preserve key data points first. Re-running the tool is a valid recovery but the diagnosis is incomplete. C wrong: Checkpoints are for crash recovery, not accessing historical data mid-loop. D wrong: If compressed at step 15, it is gone at step 22.

**Q18. Claude drafts legal contract clauses. Self-review confirms them. Human expert finds significant legal errors both passes missed. Correct architectural response?**

A) Increase self-review passes to 3 or 4

B) Replace self-review with validation retry loop

✓ **C) Implement multi-instance review — independent instances don't share legal reasoning blind spots that cause both self-review passes to validate incorrect clauses**

D) Add more few-shot examples of correct legal clauses

**Explanation:**

- C correct: Self-review fails because both passes share the same legal knowledge gaps. Multi-instance review: independent instances reason from scratch without anchoring to previous output. Disagreement triggers legal expert review.
- A wrong: More self-review passes all share the same blind spots. B wrong: Retry loops fix structural problems (invalid JSON), not semantic legal errors. D wrong: Few-shot teaches format consistency — doesn't substitute for legal reasoning.

**Q19. Customer data platform: should customer lookup tools (read-only) and order management tools (write-enabled) be in the same or separate MCP servers?**

A) Always separate — each business domain needs its own server

B) Always together — simpler architecture

✓ **C) Deciding factor is permission scope: different credentials and permission levels → separate servers to enforce minimal footprint isolation**

D) Deciding factor is team ownership

**Explanation:**

- C correct: Permission scope and credential isolation determine server separation. Read-only vs write-enabled require different credentials. Separate servers: lookup server physically cannot write orders.
- A wrong: Business domain ≠ permission boundary. Same-permission domains can coexist. B wrong: Simplicity never justifies over-permissioning. D wrong: Team ownership is organisational — permission boundaries are architectural.

**Q20. 5-agent sequential pipeline: 1=ingest, 2=extract, 3=analyse, 4=generate report, 5=review. Is this optimal? What would you change?**

A) Optimal — sequential pipeline always correct for financial data

B) Steps 1-4 should be parallelised, step 5 reviews all simultaneously

✓ **C) Mostly correct — steps 1-4 sequential dependencies are real. But step 5 should use multi-instance review with independent instances rather than single review agent, given high-stakes financial context**

D) Steps 2 and 3 should be merged — data extraction and analysis are the same operation

**Explanation:**

- C correct: Steps 1-4 are correctly sequential (each depends on previous output). Step 5 weakness: single review agent shares blind spots with production agents. Fix: multi-instance review with independent instances — disagreement escalates to human.
- A wrong: Sequential for 1-4 is correct but review step is suboptimal. B wrong: Steps 1-4 have hard dependencies — parallelisation is architecturally impossible. D wrong: Extraction and analysis are distinct operations with different outputs.

## Overall certification readiness

| Mock exam            | Score        | Readiness  | Buffer above 72%  |
|----------------------|--------------|------------|-------------------|
| Mock Exam 1 (Day 14) | 19/20 = 95%  | <b>88%</b> | <b>+16 points</b> |
| Mock Exam 2 (Day 17) | 20/20 = 100% | <b>92%</b> | <b>+20 points</b> |
| Pass threshold       | —            | 72%        | —                 |

**You are exam ready**

- Mock Exam 1: 95% (19/20). Mock Exam 2: 100% (20/20).
- Estimated readiness: 92% — 20 points above the 72% pass threshold.
- Only concept that caught you across both mocks: critical path calculation (Mock 1, Q5).
- Formula locked in: Total = MAX(parallel agents) + SUM(sequential steps after).
- Day 18 is final revision — key concepts, common traps, and exam-day tips.

**Next up: Day 18**

Final revision — key concepts, most common exam traps, and exam-day strategy.